Actividad 1.2
Tema 1. Almacenamiento de la información
Base de datos 26/27
Daniel Jiménez Carrasco

1. Investigue y explique con sus palabras en qué consiste el método de búsqueda binaria en ficheros.

Consiste que en vez de revisar un archivo gigantesco de principio a fin, la búsqueda binaria va dividiendo el problema por la mitad a cada paso, logrando encontrar cualquier dato en muy pocos accesos al disco.

2. Realice un ejemplo de búsqueda secuencial y binaria en clase suponiendo que tiene que acceder a un valor dentro de un conjunto ordenado de valores. Compute y compare el número de lecturas en ambos procesos para varios valores de búsqueda.

Supongamos un conjunto ordenado de 10 elementos representados en un fichero de registros:

Fichero = [5, 12, 18, 24, 31, 40, 52, 67, 79, 88]

Búsqueda Secuencial: Recorre el conjunto elemento por elemento desde el inicio.

Búsqueda Binaria: Comienza en la posición central m = (inicio + fin)/2

Pruebas con varios valores:

1.Buscar valor 12:

Secuencial: Lee 5 (1.ª lectura), luego 12 (2.ª lectura). Total: 2 lecturas.

Binaria:

a.Compara con la posición media index 4 (valor 31)-> 12 < 31, busca a la izquierda.

b. Compara con la posición media index 1 (valor 12)-> ¡Encontrado!

Total de lecturas: 2

2.Buscar valor 31:

Secuencial: Lee 5, 12, 18, 24, 31. Total: 5 lecturas.

Binaria:

a. Compara con la posición media index 4 (valor 31)-> ¡Encontrado en el primer intento!

Total de lectura: 1

3.Buscar valor 79:

Secuencial: Lee 5, 12, 18, 24, 31, 40, 52, 67, 79. Total: 9 lecturas.

Binaria:

a. Compara con index 4 (valor 31)-> 79 > 31, busca a la derecha (índices 5 a 9).

b. Compara con index 7 (valor 67)-> 79 > 67, busca a la derecha (índices 8 a 9).

c. Compara con index 8 (valor 79)-> ¡Encontrado!

Total de lecturas: 3

3. ¿Cuántos índices primarios y de agrupamiento puede tener un fichero ordenado?

Un índice primario puede tener como máximo 1 y un índice de agrupamiento puede tener también como máximo 1.

4. Comente ventajas e inconvenientes respecto a la actualización de datos en ficheros con organización tipo hash.

Ventajas respecto a la actualización de datos en ficheros con organización tipo hash:

- Modificación directa: Si la clave principal no cambia, actualizar los campos de un registro existente es extremadamente rápido
- Inserción rápida sin reordenamiento: A diferencia de los ficheros ordenados, al insertar un registro nuevo no es necesario desplazar físicamente otros registros para mantener un orden.
- Independencia de registros: Las actualizaciones en un cubo no afectan a los bloques adyacentes.

Inconvenientes respecto a la actualización de datos en ficheros con organización tipo hash:

- Gestión de desbordamientos: Si al insertar o actualizar un registro el tamaño aumenta o la cubeta de destino está llena, se requiere usar áreas de desbordamiento, lo que degrada drásticamente el rendimiento de futuras consultas y actualizaciones.

- Modificación de la clave hash: Si la actualización implica cambiar el valor del campo que actúa como clave Hash, el registro debe eliminarse de su cubeta original y reinsertarse en una cubeta completamente distinta.
- Reorganización / Rehash costoso: En organizaciones Hash estáticas, si el número de actualizaciones/inserciones hace crecer mucho el fichero, el factor de empaquetado cae o las cadenas de desbordamiento se vuelven muy largas, requiriendo un proceso de rehash global altamente costoso.

5. Si tenemos un archivo de datos de 2.000 jugadores con tamaño fijo de 80 bytes y un disco de tamaño de bloque igual a 1.024 bytes, determine el número de bloques requerido y el coste de una búsqueda binaria en cuanto a número necesario de accesos a bloques para encontrar un registro de datos.

Datos iniciales:

- Número de jugadores (N) = 2.000 registros
- Tamaño de cada jugador (R) = 80 bytes
- Tamaño de un bloque de disco (B) = 1.024 bytes

Paso 1: Para saber cuántos registros caben en un bloque de disco, dividimos el tamaño del bloque entre lo que ocupa cada registro:

Registros por bloque = 1024 / 80 = 12,8 = 12 registros

Paso 2: Dividimos el total de jugadores entre los que caben en cada bloque:

Número de bloques = 2000 / 12 = 166,66 = 167 bloques requeridos

Paso 3: El número máximo de lecturas de bloques que necesitamos en una búsqueda binaria viene dado por el logaritmo en base 2 del número total de bloques:

Número de acceso = [log2(167)]
[log2(167)]aproximado es = 7,38 -> 8 accesos a bloque

Resultado final:
Bloques requeridos: 167 bloques
Coste de la búsqueda binaria: 8 acceso a bloques